

Taller 1

Primer punto

Curso CSS <https://flukeout.github.io/>

Level 1-8

```
p selects all p elements.
```

```
div selects all div elements.
```

```
#cool selects any <element> with id="cool"
```

```
ul#long selects ul id="long"
```

```
p strong selects all strong elements that are inside of any p
```

```
#fancy span selects any <span> elements that are inside of the
```

```
#cool span selects all <span> elements that are inside of elemen
```

```
.neato selects all elements with class="neato"
```

```
ul.important selects all ul elements that have class="important"
```

```
#big.wide selects all elements with id="big" that also have clas
```

Level 9-32

```
p, .fun selects all <p> elements as well as all elements with ci
```

```
a, p, div selects all <a>, <p> and <div> elements
```

```
p * selects any element inside all <p> elements.
```

```
ul.fancy * selects every element inside all <ul> class="fancy" e
```

`p + .intro` selects every element with `class="intro"` that directly follows a `div`.

`A ~ B` selects all `B` that follow a `A`.

`A > B` selects all `B` that are a direct children of `A`.

`:first-child` selects all first child elements.

`p:first-child` selects all first child `<p>` elements.

`div p:first-child` selects all first child `<p>` elements that are direct children of `div`.

`span:only-child` selects the `<span>` elements that are the only child of their parent.

`ul li:only-child` selects the only `<li>` element that are in a `<ul>`.

`:last-child` selects all last-child elements.

`span:last-child` selects all last-child `span` elements.

`ul li:last-child` selects the last `li` elements inside of any `ul`.

`:nth-child(8)` selects every element that is the 8th child of any parent.

`div p:nth-child(2)` selects the second `p` in every `div`.

`:nth-last-child(2)` selects all second-to-last child elements.

`span:first-of-type` selects the first `<span>` in any element.

`div:nth-of-type(2)` selects the second instance of a `div`.

.example:`:nth-of-type(odd)` selects all odd instances of a the example.

`span:nth-of-type(6n+2)` selects every 6th instance of a `span`, starting from the 2nd.

`p span:only-of-type` selects a span within any p if it is the only

`div:last-of-type` selects the last div in every element.

`p span:last-of-type` selects the last span in every p.

`div:empty` selects all empty div elements.

`:not(#fancy)` selects all elements that do not have `id="fancy"`.

`div:not(:first-child)` selects every `<div>` that is not a first child

`:not(.big, .medium)` selects all elements that do not have class `big` or `medium`.

`a[href]` selects all `<a>` elements that have a `href="anything"` attribute.

`[type]` selects all elements that have a `type="anything"` attribute.

`[value]` selects all elements that have a `value="anything"` attribute.

`a[href]` selects all `<a>` elements that have a `href="anything"` attribute.

`input[disabled]` selects all `<input>` elements with the `disabled` attribute.

`input[type="checkbox"]` selects all checkbox input elements.

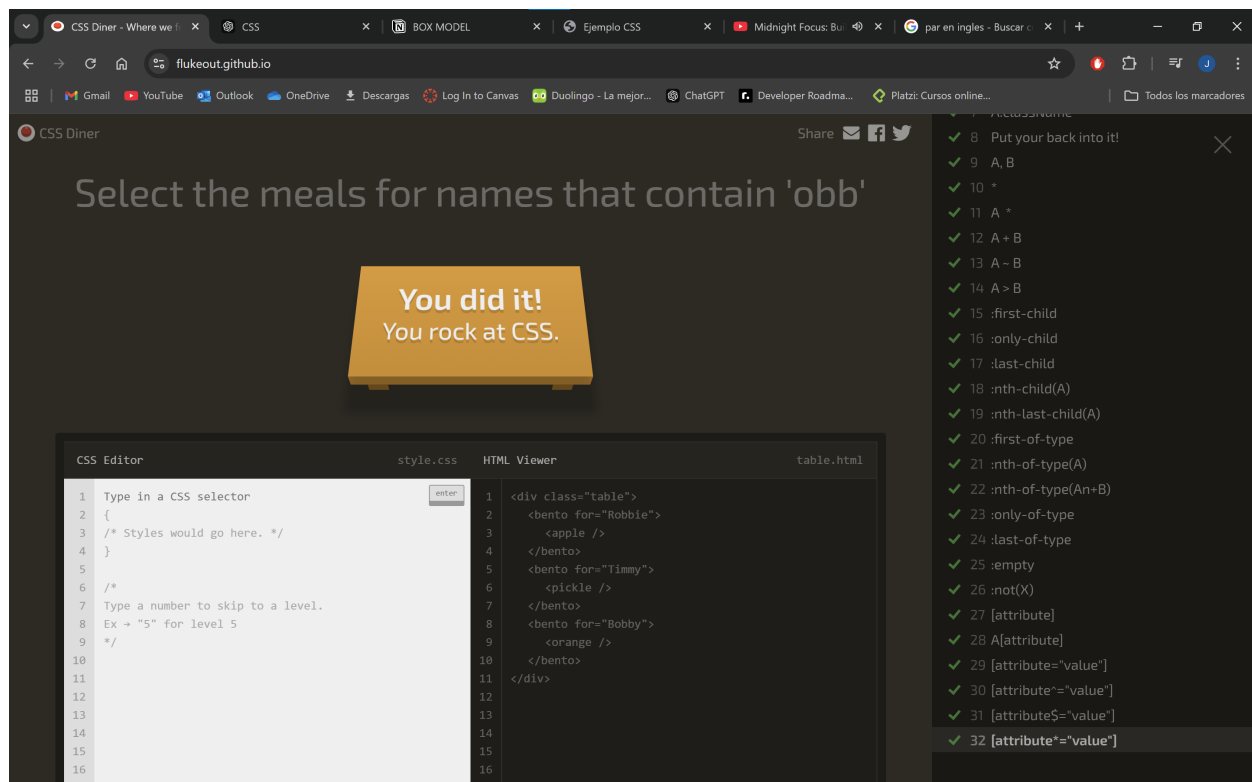
`.toy[category^="Swim"]` selects elements with class `toy` and either `category="Swim"` or `category="Swim "`.

`img[src$=".jpg"]` selects all images that display a .jpg image.

`img[src*="/thumbnails/"]` selects all image elements that show images in thumbnails.

`[class*="heading"]` selects all elements with "heading" in their class name.

Evidencia de culminación del primer curso



Flexbox: <https://flexboxfroggy.com/#es>

Flexbox Froggy

Nivel 1 de 24

Bienvenido a Flexbox Froggy, un juego donde ayudarás a Froggy y a sus amigos escribiendo código CSS. Guía a esta rana hacia la hoja de lirio en la derecha, usando la propiedad `justify-content`, la cual alinea elementos horizontalmente y acepta los siguientes valores:

- `flex-start` : Alinea elementos al lado izquierdo del contenedor.
- `flex-end` : Alinea elementos al lado derecho del contenedor.
- `center` : Alinea elementos en el centro del contenedor.
- `space-between` : Muestra elementos con la misma distancia entre ellos.

- `space-around` : Muestra elementos con la misma separación alrededor de ellos.

Por ejemplo, `justify-content: flex-end;` moverá la rana a la derecha.

Ahora usa `align-items` para ayudar a las ranas a llegar al fondo del estanque. Esta propiedad CSS alinea elementos verticalmente y acepta los siguientes valores:

- `flex-start` : Alinea elementos a la parte superior del contenedor.
- `flex-end` : Alinea elementos a la parte inferior del contenedor.
- `center` : Alinea elementos en el centro (verticalmente hablando) del contenedor.
- `baseline` : Muestra elementos en la línea base del contenedor
- `stretch` : Elementos se estiran para ajustarse al contenedor.

Las ranas necesitan ponerse en el mismo orden que sus hojas de lirio usando `flex-direction` . Esta propiedad CSS define la dirección de los elementos en el contenedor, y acepta los siguientes valores:

- `row` : Elementos son colocados en la misma dirección del texto.
- `row-reverse` : Elementos son colocados en la dirección opuesta al texto.
- `column` : Elementos se colocan de arriba hacia abajo.
- `column-reverse` : Elementos se colocan de abajo hacia arriba.

A veces, invertir el orden de una fila o columna de un contenedor no es suficiente. En esos casos, nosotros podemos aplicar la propiedad `order` a elementos individuales. Por defecto, los elementos tienen un valor 0, pero nosotros podemos usar esta propiedad para establecerlo a un número entero positivo o negativo.

Usa la propiedad `order` para reordenar las ranas de acuerdo a sus hojas de lirio.

Otra propiedad que puedes aplicar a elementos individuales es `align-self`. Esta propiedad acepta los mismos valores de `align-items` y sus valores son usados para un elemento específico

¡Oh, no! Todas las ranas están apretadas en una sola fila de hojas de lirio. Manos a la obra, vamos a distribuir las ranas correctamente usando la propiedad `flex-wrap`, la cual acepta los siguientes valores:

- `nowrap` : Cada elemento se ajusta en una sola línea.
- `wrap` : los elementos se envuelven alrededor de líneas adicionales.
- `wrap-reverse` : Los elementos se envuelven alrededor de líneas adicionales en reversa.

Las dos propiedades `flex-direction` y `flex-wrap` son usadas a menudo en conjunto con la propiedad abreviada `flex-flow`, la cual fue creada para combinarlas. Esta propiedad abreviada, acepta un valor de cada una separada por un espacio.

Por ejemplo, puedes usar `flex-flow` para establecer filas y envolverlas.

Trata de usar `flex-flow` para volver a realizar el nivel anterior.

`flex-flow: column wrap;`

Las ranas están repartidas por todo el estanque, pero las hojas de lirio están agrupadas en la parte superior. Puedes usar `align-content` para establecer como múltiples líneas están separadas una de otra. Esta propiedad acepta los siguientes valores:

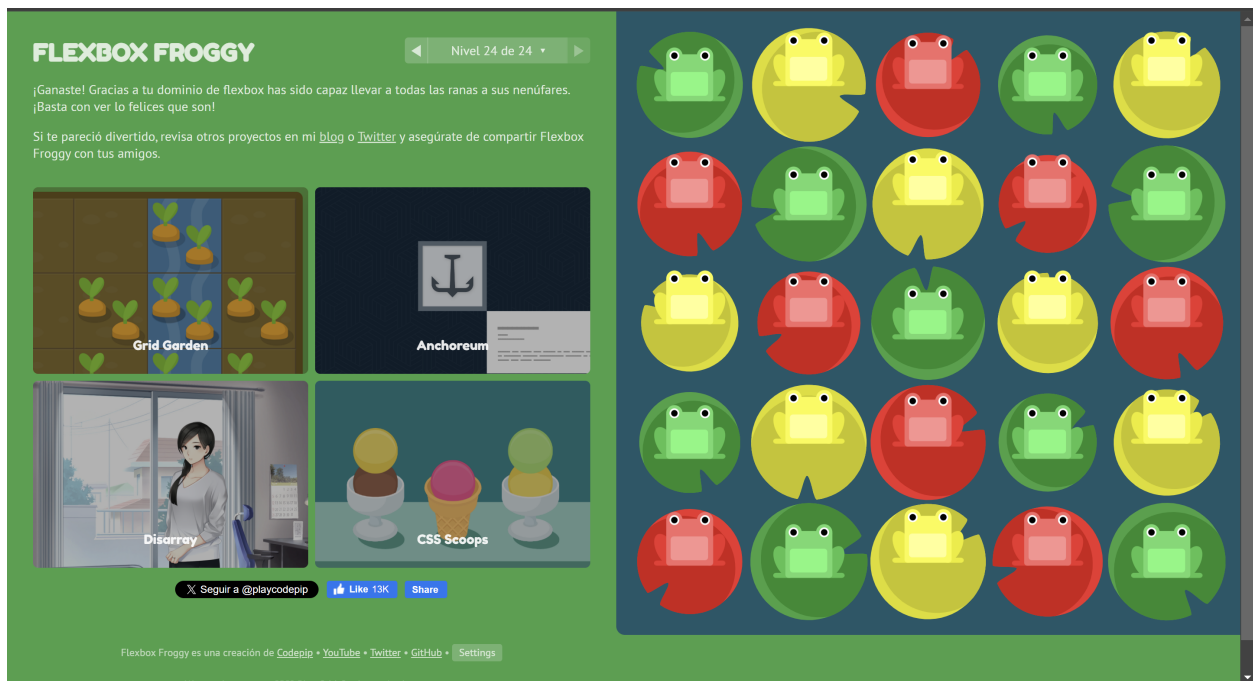
- `flex-start` : Las líneas se posicionan en la parte superior del contenedor.
- `flex-end` : Las líneas se posicionan en la parte inferior del contenedor.
- `center` : Las líneas se posicionan en el centro (verticalmente hablando) del contenedor.
- `space-between` : Las líneas se muestran con la misma distancia entre ellas.

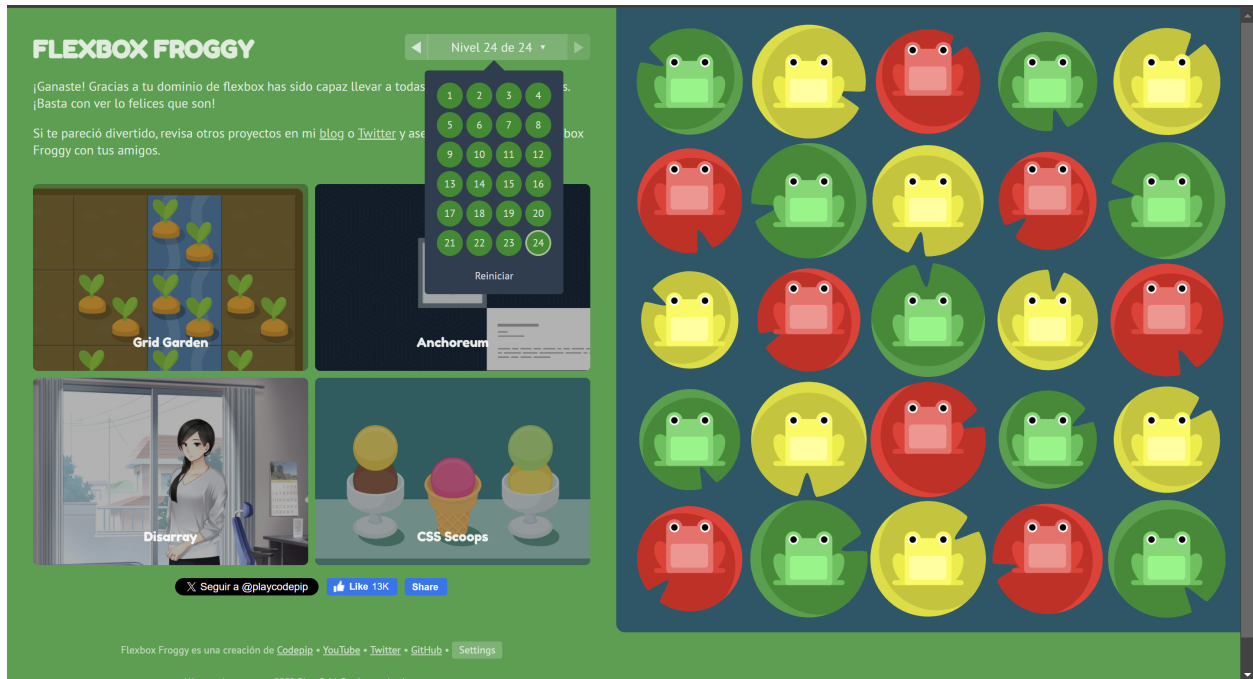
- **space-around** : Las líneas se muestran con la misma separación alrededor de ellas.
- **stretch** : Las líneas se estiran para ajustarse al contenedor.

Esto puede ser confuso, pero **align-content** determina el espacio entre las líneas, mientras que **align-items** determina como los elementos en su conjunto están alineados dentro del contenedor. Cuando hay solo una línea, **align-content** no tiene efecto.

Ejercicio 24:

```
flex-wrap: wrap-reverse;
flex-direction: column-reverse;
justify-content: center;
align-content: space-between;
```





gridlayout: Desarrollar los retos de gridgarden <https://cssgridgarden.com/#es>

Grid Garden

Nivel 1 de 28

¡Bienvenido a Grid Garden, donde escribirás tu código CSS para cultivar tu jardín de zanahorias! Riega solo las áreas que tienen zanahorias usando la propiedad `grid-column-start`.

Por ejemplo, `grid-column-start: 3;` regará el área comenzando por la tercera línea vertical, que es otra manera de decir el 3er borde vertical contando desde la izquierda de la cuadrícula.

Cuando `grid-column-start` se usa solo, la expansión por defecto del elemento en la cuadrícula será de exactamente una columna. No obstante, puedes extender el elemento varias columnas añadiendo la propiedad `grid-column-end`.

Usando `grid-column-end`, riega todas las zanahorias evitando que se forme barro. ¡No queremos malgastar agua! Fíjate en que las zanahorias comienzan en el 1er borde vertical y terminan en el 4º.

Si prefieres contar las líneas de la cuadrícula comenzando por la derecha, puedes dar a `grid-column-start` y `grid-column-end` valores negativos. Por ejemplo, puedes establecerlos a -1 para indicar la primera línea comenzando por la derecha.

Intenta establecer `grid-column-end` a un valor negativo.

En lugar de definir un elemento en la cuadrícula basado en la posición inicial y final, puedes definirlo basado en la longitud de columnas deseada usando la palabra clave `span`. Ten presente que `span` solo funciona con valores positivos.

Por ejemplo, riega las zanahorias usando `grid-column-end: span 2;`.

También puedes usar la palabra clave `span` con `grid-column-start` para establecer la anchura del elemento en relación a la posición final.

Escribir ambos `grid-column-start` y `grid-column-end` cada vez puede resultar cansador. Afortunadamente, `grid-column` es una propiedad abreviada que acepta ambos valores a la vez, separados por una barra oblicua.

Por ejemplo, `grid-column: 2 / 4;` establecerá el comienzo del elemento de la cuadrícula en la 2ª línea vertical de esta, y su final en la 4ª línea vertical.

Intenta usar `grid-column` para regar las zanahorias. La palabra clave `span` también funciona con esta propiedad abreviada así que ¡dale una oportunidad!

Una de las cosas que diferencia las cuadrículas de CSS de flexbox es que puedes posicionar los elementos fácilmente en 2 dimensiones: columnas y filas. `grid-row-`

`start` funciona de manera semejante a `grid-column-start` pero a lo largo del eje vertical.

Usa `grid-row-start` para regar las zanahorias.

También puedes usar `grid-column` y `grid-row` juntos para abarcar áreas más extensas en la cuadrícula. ¡Inténtalo!

Si escribir `grid-column` y `grid-row` se te hace demasiado pesado, aquí tienes otra propiedad abreviada. `grid-area` admite cuatro valores separados por barras oblicuas: `grid-row-start`, `grid-column-start`, `grid-row-end`, seguido de `grid-column-end`.

Un ejemplo de esto podría ser `grid-area: 1 / 1 / 3 / 6;`.

¿Y qué me dices de múltiples elementos? Puedes superponerlos sin problema. Usa `grid-area` para definir una segunda área que cubra todas las zanahorias que están sin regar.

Si los elementos de la cuadrícula no se sitúan explícitamente con `grid-area`, `grid-column`, `grid-row`, etc., se sitúan automáticamente de acuerdo al orden en el código fuente. Puedes sobrescribir esto usando la propiedad `order`, que es una de las ventajas de la cuadrícula frente al diseño basado en tablas.

Por defecto, el valor de `order` de todos los elementos es igual a 0, pero puede ser establecido a cualquier valor positivo o negativo, de manera similar a `z-index`.

Ahora mismo, las zanahorias en la segunda columna están siendo envenenadas y las malas hierbas en la última columna, regadas. ¡Cambia el valor de `order` del veneno para solucionar esto!

Hasta este momento, has tenido un jardín formado por cinco columnas, cada una ocupando el 20% de la anchura total, y cinco filas, cada una ocupando el 20% de la altura total.

Esto ha sido establecido con las propiedades `grid-template-columns: 20% 20% 20% 20% 20%;` y `grid-template-rows: 20% 20% 20% 20% 20%;`. Cada propiedad tiene cinco valores que crean cinco columnas, cada una establecida al 20% de la anchura total del jardín.

Pero puedes establecer los valores en la cuadrícula como quieras. Da a `grid-template-columns` un nuevo valor para regar tus zanahorias. Querrás que la anchura de la primera columna sea del 50%.

Especificar un puñado de columnas con la misma anchura puede ser aburrido. Afortunadamente hay una función `repeat` que te ayudará con eso.

Por ejemplo, previamente hemos definido cinco columnas al 20% de anchura mediante `grid-template-columns: 20% 20% 20% 20% 20%;`. Esto puedes simplificarse como `grid-template-columns: repeat(5, 20%);`

Usando `grid-template-columns` con la función `repeat`, crea ocho columnas, cada una con una anchura del 12.5%. De esta manera no inundarás tu jardín.

CSS Grid también introduce una nueva medida, la fracción `fr`. Cada unidad `fr` asigna una porción del espacio disponible. Por ejemplo, si dos elementos están establecidos a `1fr` y `3fr` respectivamente el espacio se divide en 4 porciones iguales; el primer elemento ocupa 1/4 del espacio y el segundo elemento los 3/4 restantes.

Ahora las malas hierbas ocupan 1/6 de tu primera fila y las zanahorias los restantes 5/6. Crea dos columnas con ese ancho empleando `fr`.

rt: `grid-template-columns: 1fr 5fr`

Cuando algunas columnas son establecidas en píxeles, porcentajes o ems, cualquier otra columna establecida con `fr` dividirá el espacio restante.

Aquí las zanahorias forman una columna de 50 píxeles a la izquierda, y las malas hierbas una columna de 50 píxeles a la derecha. Mediante `grid-template-columns`, crea esas dos columnas y usa `fr` para crear tres columnas más en el espacio que queda entre ellas.

```
rt: grid-template-columns: 50px 1fr 1fr 1fr 50px;
```

Ahora hay una columna de malas hierbas de 75 píxeles en el lado izquierdo del jardín. En 3/5 del espacio restante crecen zanahorias, mientras que los 2/5 restantes han sido invadidos por malas hierbas.

Usa `grid-template-columns` con una combinación de `px` y `fr` para crear las columnas necesarias.

```
rt: grid-template-columns: 75px 3fr 2fr
```

`grid-template-rows` funciona exactamente igual que `grid-template-columns`.

Usa `grid-template-rows` para regar todo excepto los 50 píxeles de la parte superior de tu jardín. Fíjate que el agua se ha establecido para que llene solo la 5ª fila, por lo que tendrás que crear 5 filas en total.

```
rt: grid-template-rows: 12.5px 12.5px 12.5px 12.5px;
```

`grid-template` es una propiedad abreviada que combina `grid-template-rows` y `grid-template-columns`.

Por ejemplo, `grid-template: 50% 50% / 200px;` creará una cuadrícula con dos filas que ocuparán el 50% del alto cada una, y una columna que será 200 píxeles de ancho.

Prueba a usar `grid-template` para regar un área que incluya el 60% superior y 200 píxeles desde la izquierda en tu jardín.

```
rt: grid-template: 60% 40% / 200px;
```

```
rt: grid-template: 1fr 50px/ 20% 1fr
```

